

Program 6: Singly Linked List: Reverse

Aim

To write a C program that reverses a singly linked list.

Theory

A linked list node holds a **data** field and a **next** pointer; **head** points to the first node and the last node points to NULL.

Reversing a linked list means reversing the direction of every next pointer, so the last node becomes the new head. Only pointers change - data is not moved.

Iterative method uses three pointers (prev, curr, next). At each step: save next = curr->next, set curr->next = prev, then advance prev = curr and curr = next. When curr is NULL, prev is the new head. Time O(n), space O(1).

Recursive method: reverse the rest of the list, then make the second node point back to the first node and set the first node's next to NULL. Time O(n), space O(n).

Algorithm

1. Initialise prev = NULL and curr = head.
2. While curr != NULL: next = curr->next; curr->next = prev; prev = curr; curr = next.
3. Set head = prev.
4. Recursive version: if head is NULL or head->next is NULL return head; newHead = reverse(head->next); head->next->next = head; head->next = NULL; return newHead.
5. Display the list before and after reversing.

C Source Code

```
/*
 * Program : Singly Linked List - Reverse
 *
 * THEORY:
 * - Reversing a linked list means reversing the direction of every next
 *   pointer, so the last node becomes the head. Data is NOT moved.
 * - Iterative method uses three pointers:
 *   prev = NULL, curr = head, next
 *   loop: next = curr->next; curr->next = prev; prev = curr; curr = next;
 *   At the end prev is the new head. Time O(n), space O(1).
 * - Recursive method: reverse the rest of the list, then make the second
 *   node point back to the first. Time O(n), space O(n) (call stack).
 */
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *insertEnd(struct Node *head, int val) {
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->data = val;
    n->next = NULL;
    if (head == NULL) return n;
    struct Node *t = head;
    while (t->next != NULL) t = t->next;
    t->next = n;
}
```

```

        return head;
    }

void display(struct Node *head) {
    for (struct Node *t = head; t != NULL; t = t->next)
        printf("%d -> ", t->data);
    printf("NULL\n");
}

struct Node *reverseIterative(struct Node *head) {
    struct Node *prev = NULL, *curr = head, *next = NULL;
    while (curr != NULL) {
        next = curr->next;    /* save next node */
        curr->next = prev;    /* reverse the link */
        prev = curr;         /* move prev forward */
        curr = next;         /* move curr forward */
    }
    return prev;             /* new head */
}

struct Node *reverseRecursive(struct Node *head) {
    if (head == NULL || head->next == NULL)
        return head;
    struct Node *newHead = reverseRecursive(head->next);
    head->next->next = head;    /* make next node point back */
    head->next = NULL;
    return newHead;
}

int main() {
    struct Node *head = NULL;
    int n, val;

    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter %d values: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        head = insertEnd(head, val);
    }

    printf("\nOriginal list      : "); display(head);
    head = reverseIterative(head);
    printf("After iterative reverse : "); display(head);
    head = reverseRecursive(head);
    printf("After recursive reverse  : "); display(head);

    while (head != NULL) { struct Node *t = head; head = head->next; free(t); }
    return 0;
}

```

Sample Input

Input: 5 nodes with values 1 2 3 4 5.

Sample Output (program run with the input above)

```

Enter number of nodes: Enter 5 values:
Original list      : 1 -> 2 -> 3 -> 4 -> 5 -> NULL
After iterative reverse : 5 -> 4 -> 3 -> 2 -> 1 -> NULL
After recursive reverse : 1 -> 2 -> 3 -> 4 -> 5 -> NULL

```